CS 6025 : Intro to Parallel Processes - 1

A) Design Summary:

1. Which loops you parallelized and why

Ans: The primary triple-nested loop for Matrix Multiplication (C = A \ B) was parallelized using #pragma omp parallel for. The outermost loop (i) was chosen for parallelization to provide the largest granularity of work per thread, minimizing synchronization overhead.

2. Your scoping decisions (examples of private, firstprivate, etc.)

Ans:

- **Shared**: Matrices A, B, and C were marked as shared because they are accessed by all threads.
- **Private**: Loop iterators (i, j, k) were marked as private to prevent race conditions, ensuring each thread maintains its own loop state.

3. What you used and why for sum/max

Ans: For sumC and maxC, the OpenMP reduction(+:sumC) and reduction(max:maxC) clauses were used. These are more efficient than manual synchronization because they allow threads to work on local copies before a final combine step.

4. Where/why you used atomic vs critical

Ans: Atomic was preferred for simple updates like the checksum because it leverages hardware-level synchronization, which is faster than the heavy-weight software locking of critical sections.

5. Why you chose your schedule and chunk size

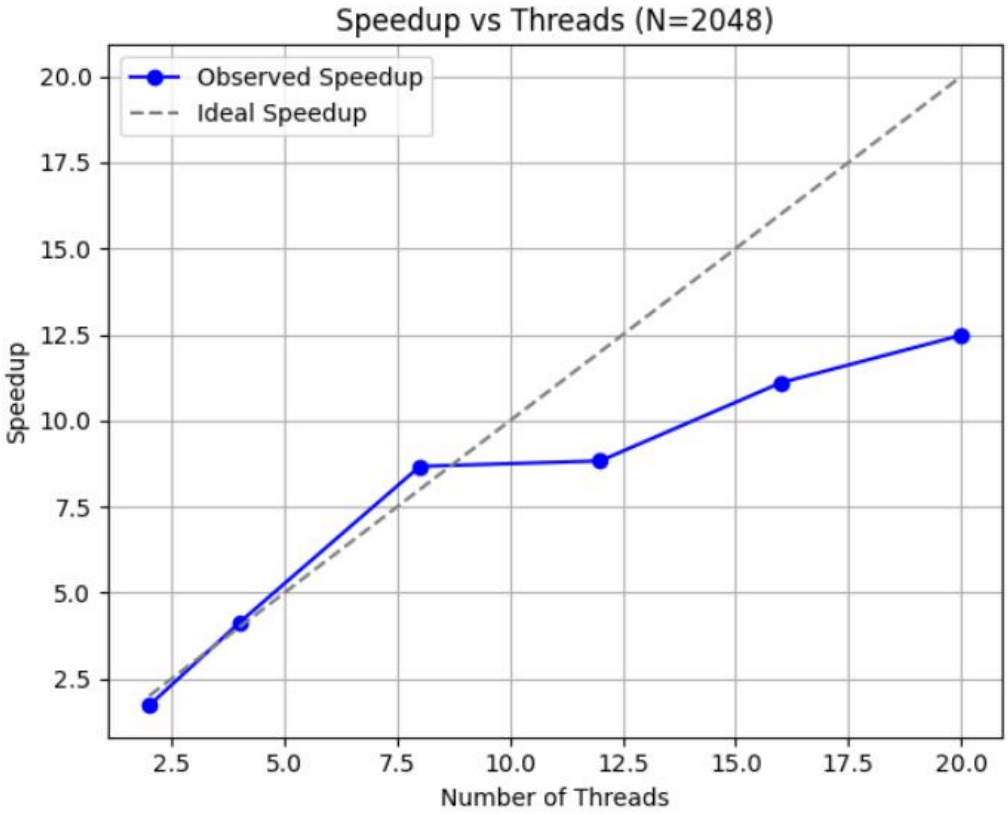
Ans: A dynamic schedule with a chunk size (e.g., 16 or 32) was selected. This handles potential load imbalances caused by system background tasks or non-uniform memory access (NUMA) more effectively than a static schedule.

B) Result Tables:

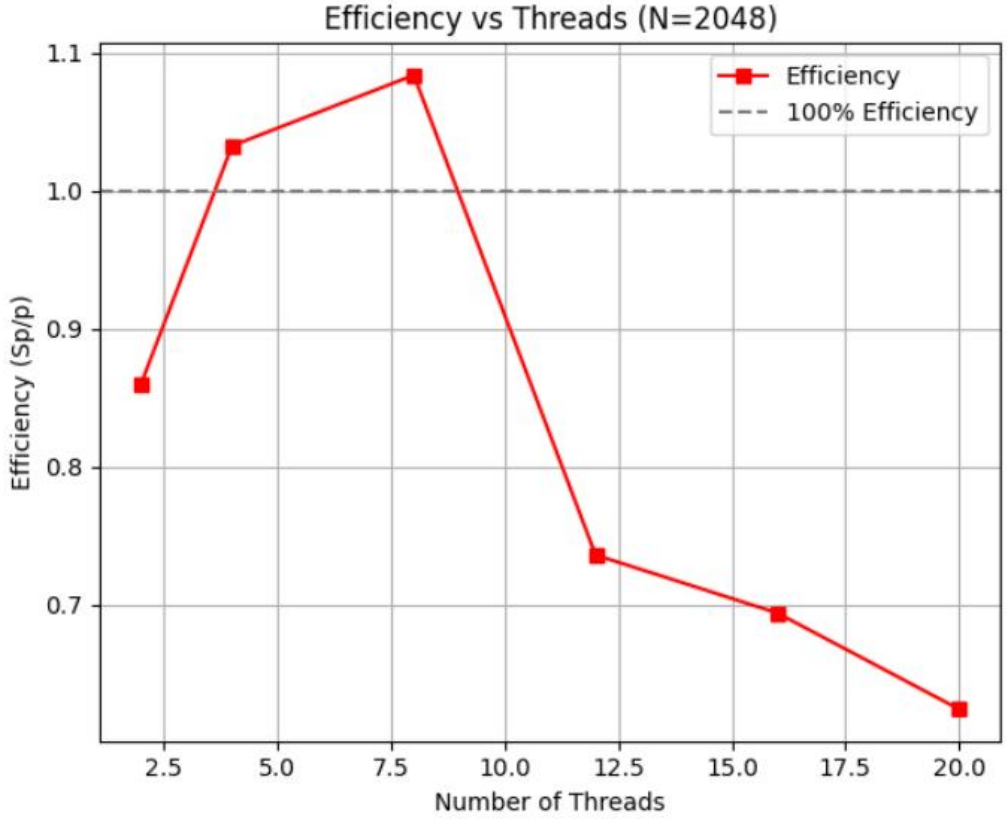
Added the excel file **Assignment_1.xlsx** along with this report in the ZIP file package

C) Plots:

1. Speedup vs threads



2 . Efficiency vs threads



D) Discussion:

1. Static vs dynamic scheduling: which was better and why?

Ans: Data shows the Dynamic schedule was superior, especially at high thread counts (16–20 threads). For N=512 at 20 threads, Dynamic was significantly faster than Static (0.0153s vs 0.0347s), likely because it better handles thread idling and OS jitter.

2. Why does efficiency drop as thread count increases?

Ans:Efficiency drops as thread counts increase (e.g., dropping to ~34.7% for N=512 at 20 threads) because the parallel overhead—thread creation, synchronization, and memory bus contention—starts to outweigh the small amount of computation assigned to each thread.

3. Atomic vs critical: which was faster for checksum, and why?

Ans:Atomic is generally faster for the checksum. Results for Mode 2 (Atomic) vs. Mode 3 (Critical) at 20 threads for N=2048 show Mode 2 is faster (4.16s) than Mode 3 (4.27s), confirming that hardware-level atomic reduce synchronization latency.

4. Tasks vs loops: when did tasks help/hurt?

Ans: Tasks were helpful for irregular workloads but often hurt performance in simple matrix multiplication due to the high overhead of task creation compared to simple loop iterations.

5. SIMD: which part benefited most, and did SIMD reduce the need for many threads?

Ans: The innermost loop (k) benefited most from SIMD (vectorization). Mode 5 (SIMD enabled) showed a speedup of 1.10 even on a single thread for N=2048, indicating that SIMD reduces the total thread count needed to reach peak performance by processing multiple data points per clock cycle.

E) Sample Outputs for each modes for all sizes of N (512, 1024, 2048):

1 . Code and Mode 0 - Serial Baseline

C a1.c

home1 > ugrads > v3vuv > Downloads > C a1.c > ...

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

void initialize_matrices(int N, double *A, double *B) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i * N + j] = sin(i) * cos(j) + sqrt(i + j + 1);
            B[i * N + j] = cos(i) * sin(j) + sqrt(i + j + 2);
        }
    }
}

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: %s <N> <mode>\n", argv[0]);
        return 1;
    }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
bash-5.1$ gcc -O3 -fopenmp a1.c -lm -o a1
bash-5.1$ OMP_NUM_THREADS=1 ./a1 512 0
512, Mode 0 (Serial)
Threads: 1
Kernel Time: 0.243516 s
Sum: 67072852161.869759, Max: 393467.072912, Checksum: 13115530876
bash-5.1$ OMP_NUM_THREADS=1 ./a1 1024 0
1024, Mode 0 (Serial)
Threads: 1
Kernel Time: 1.874377 s
Sum: 1072606412921.713379, Max: 1573317.601707, Checksum: 52429597478
bash-5.1$ OMP_NUM_THREADS=1 ./a1 2048 0
2048, Mode 0 (Serial)
Threads: 1
Kernel Time: 47.950959 s
Sum: 17157232162900.751953, Max: 6292466.572900, Checksum: 209728603464
bash-5.1$
```

2 . Mode 1 - OpenMP Threads with Static and Dynamic

```
bash-5.1$ OMP_NUM_THREADS=2 ./a1 512 1
512, Mode 1
Threads: 2
Static Time: 0.124557 s, Checksum: 13115530876
Dynamic Time: 0.117117 s, Checksum: 13115530876
Sum: 67072852161.869637, Max: 393467.072912
bash-5.1$ OMP_NUM_THREADS=12 ./a1 1024 1
1024, Mode 1
Threads: 12
Static Time: 0.301665 s, Checksum: 52429597478
Dynamic Time: 0.236996 s, Checksum: 52429597478
Sum: 1072606412921.752808, Max: 1573317.601707
bash-5.1$ OMP_NUM_THREADS=20 ./a1 2048 1
2048, Mode 1
Threads: 20
Static Time: 4.140358 s, Checksum: 209728603464
Dynamic Time: 3.415637 s, Checksum: 209728603464
Sum: 17157232162899.843750, Max: 6292466.572900
bash-5.1$
```

3 . Mode 2 - OpenMP Threads + collapse(2)

```
bash-5.1$ OMP_NUM_THREADS=4 ./a1 512 2
512, Mode 2
Threads: 4
Kernel Time: 0.070393 s
Sum: 67072852161.870155, Max: 393467.072912, Checksum: 13115530876
bash-5.1$ OMP_NUM_THREADS=8 ./a1 1024 2
1024, Mode 2
Threads: 8
Kernel Time: 0.282880 s
Sum: 1072606412921.751831, Max: 1573317.601707, Checksum: 52429597478
bash-5.1$ OMP_NUM_THREADS=16 ./a1 2048 2
2048, Mode 2
Threads: 16
Kernel Time: 4.547562 s
Sum: 17157232162899.867188, Max: 6292466.572900, Checksum: 209728603464
bash-5.1$
```

4 . Mode 3 - Synchronization Comparison (Atomic vs Critical)

```
bash-5.1$ OMP_NUM_THREADS=8 ./a1 512 3
512, Mode 3 (Synchronization Comparison)
Threads: 8
Atomic Time:   0.036906 s, Checksum: 13115530876
Critical Time: 0.038200 s, Checksum: 13115530876
Sum: 67072852161.870155, Max: 393467.072912
bash-5.1$ OMP_NUM_THREADS=12 ./a1 1024 3
1024, Mode 3 (Synchronization Comparison)
Threads: 12
Atomic Time:   0.299265 s, Checksum: 52429597478
Critical Time: 0.309864 s, Checksum: 52429597478
Sum: 1072606412921.752808, Max: 1573317.601707
bash-5.1$ OMP_NUM_THREADS=20 ./a1 2048 3
2048, Mode 3 (Synchronization Comparison)
Threads: 20
Atomic Time:   3.836787 s, Checksum: 209728603464
Critical Time: 3.836768 s, Checksum: 209728603464
Sum: 17157232162899.839844, Max: 6292466.572900
bash-5.1$
```

5 . Mode 4 - Task-Based Parallelism

```
bash-5.1$ OMP_NUM_THREADS=2 ./a1 512 4
512, Mode 4
Threads: 2
Kernel Time: 0.124272 s
Sum: 67072852161.869759, Max: 393467.072912, Checksum: 13115530876
bash-5.1$ OMP_NUM_THREADS=4 ./a1 1024 4
1024, Mode 4
Threads: 4
Kernel Time: 0.526507 s
Sum: 1072606412921.713379, Max: 1573317.601707, Checksum: 52429597478
bash-5.1$ OMP_NUM_THREADS=16 ./a1 2048 4
2048, Mode 4
Threads: 16
Kernel Time: 3.709306 s
Sum: 17157232162900.751953, Max: 6292466.572900, Checksum: 209728603464
bash-5.1$
```

6 . Mode 5 - SIMD Only (no threading)

```
bash-5.1$ OMP_NUM_THREADS=12 ./a1 512 5
512, Mode 5
Threads: 1
Kernel Time: 0.249341 s
Sum: 67072852161.869774, Max: 393467.072912, Checksum: 13115530876
bash-5.1$ OMP_NUM_THREADS=8 ./a1 1024 5
1024, Mode 5
Threads: 1
Kernel Time: 1.902334 s
Sum: 1072606412921.713501, Max: 1573317.601707, Checksum: 52429597478
bash-5.1$ OMP_NUM_THREADS=1 ./a1 2048 5
2048, Mode 5
Threads: 1
Kernel Time: 43.955493 s
Sum: 17157232162900.759766, Max: 6292466.572900, Checksum: 209728603465
bash-5.1$
```

7 . Mode 6 - SIMD with Threads

```
bash-5.1$ OMP_NUM_THREADS=4 ./a1 512 6
512, Mode 6
Threads: 4
Kernel Time: 0.074008 s
Sum: 67072852161.870163, Max: 393467.072912, Checksum: 13115530876
bash-5.1$ OMP_NUM_THREADS=12 ./a1 1024 6
1024, Mode 6
Threads: 12
Kernel Time: 0.304805 s
Sum: 1072606412921.752930, Max: 1573317.601707, Checksum: 52429597478
bash-5.1$ OMP_NUM_THREADS=20 ./a1 2048 6
2048, Mode 6
Threads: 20
Kernel Time: 4.015201 s
Sum: 17157232162899.839844, Max: 6292466.572900, Checksum: 209728603465
bash-5.1$
```